Aarya Vijay Arban
Batch – S11
Roll no. – 07

Aim – Implementation of Adjacency Matrix Creation

Code:

```
#include<stdio.h>
#include<conio.h>
void main()
{
   int a[10][10] = {0};
   int node,edge,start,end,i,j;

   printf("Enter the number of nodes/vertices : ");
   scanf("%d",&node);
   printf("\n");
   edge = (node*(node-1)/2);
   //edge = (node*(node-1));

   for(i=0;i<edge;i++){
      printf("Enter the start point : ");
      scanf("%d",&start);
      printf("Enter the end point : ");
      scanf("%d",&end);
      printf("\n");
      a[start][end]=1;
      a[end][start]=1;
```

```
    }
    for(i=0;i<node;i++){
        for(j=0;j<node;j++){
            printf("%d\t",a[i][j]);
        }
        printf("\n");
    }
}
```

Output:

```
Enter the number of nodes/vertices : 4

Enter the start point : 2
Enter the end point : 4

Enter the start point : 3
Enter the end point : 5

Enter the start point : 4
Enter the end point : 6

Enter the start point : 5
Enter the end point : 7

Enter the start point : 2
Enter the end point : 6

Enter the start point : 3
```

Enter the end point : 5

0 0 0 0

0 0 0 0

0 0 0 0

0 0 0 0

- **DFS(Depth- First- Search) Algorithm.**

DFS(graph, start_node):

create an empty stack

push start_node onto the stack

while stack is not empty:

current_node = pop stack

if current_node has not been visited:

mark current_node as visited

process current_node (e.g., print it)

for each neighbor in graph.adjacent_nodes(current_node):

if neighbor has not been visited:

push neighbor onto the stack.

BFS (Breadth-First-Search) Algorithm.

BFS(graph, start_node):

create an empty queue

enqueue start_node into the queue

mark start_node as visited

while queue is not empty:

current_node = dequeue from the queue

process current_node (e.g., print it)

for each neighbor in graph.adjacent_nodes(current_node):

if neighbor has not been visited:

mark neighbor as visited

enqueue neighbor into the queue.